Research on Testing Plan

1. Unit Testing

Method 1: JUnit 5 (Unit Testing Backbone)

- **Applicability**: TDD and regression for service layer, utilities, and domain rules
- **Pros**:
  a. Precise failure localisation; supports red-green-refactor
  b. Millisecond-level feedback; CI-friendly via *mvn test*
  c. Mature ecosystem: parameterised/conditional tests, tagging, rich assertions
- **Cons**:
  a. Not production-like: misses real network/container/thread-pool behaviours
  b. Risk of over-isolation: integration issues (serialisation, transactions, filter chains) can be missed
  c. Requires testable design: highly coupled/static are difficult to test
- **References**:
  a. https://junit.org/junit5/docs/current/user-guide/
  b. https://www.baeldung.com/junit-5
  c. https://www.thoughtworks.com/insights/articles/effective-unit-testing
  d. https://martinfowler.com/bliki/TestPyramid.html

Method 2: Mockito (Dependency Isolation & Interaction Verification)

- **Applicability**: Isolate DAO/HTTP/MQ dependencies; verify calls/params/branches
- **Pros**:
  a. Fast and stable tests by replacing slow/unstable deps
  b. Clear collaboration boundaries via *verify* (e.g., message sent/cache updated)
  c. Expressive stubs: *when/thenReturn/thenThrow* for branches and error paths
- **Cons**:
  a. Over-mocking risk: hides real integration bugs (SQL mapping, serialisation, thread context)
  b. Maintenance cost: stubs must track API/interaction changes
  c. Weak for concurrency/context: hard to cover AOP, filters, *TransmittableThreadLocal*
- **References**:
  a. https://site.mockito.org/
  b. https://www.baeldung.com/mockito-series
  c. https://martinfowler.com/articles/mocksArentStubs.html

d. https://stackoverflow.blog/2022/01/03/best-practices-for-mocking-in-unit-tests/

Method 3: AssertJ (Fluent Assertions)

- **Applicability**: Rich, readable assertions for collections/exceptions/time/JSON
- **Pros**:
  a. Fluent API and helpful failure messages
  b. Broad coverage: collections, Optional, exceptions, time, SoftAssertions
- **Cons**:
  a. Extra dependency and learning curve
  b. Style consistency required with other assertion styles
- **References**:
  a. https://assertj.github.io/doc/
  b. https://www.baeldung.com/assertj

2. API Testing

Method 1: Spring MockMvc (Controller-layer Isolation)

- **Applicability**: MVC chain without real sockets: controllers/interceptors/filters/AOP
- **Pros**:
  a. Close to Spring MVC; covers *JwtAuthenticationTokenFilter*, CORS, exception handling
  b. Fast; friendly for header/serialisation assertions
  c. *@WebMvcTest* focuses on the web slice for faster loading
- **Cons**:
  a. Not real container behaviour: timeouts/backpressure not covered
  b. More setup: security, converters, and aspects may need explicit config
- **References**:
  a. https://docs.spring.io/spring-framework/docs/5.3.x/reference/html/testing.html#spring-mvc-test-framework
  b. https://www.baeldung.com/integration-testing-in-spring
  c. https://reflectoring.io/spring-boot-web-controller-test/

Method 2: RestAssured (HTTP-style DSL)

- **Applicability**: End-to-end API assertions against embedded or external server; JWT

flows

- **Pros**:
  a. Natural chained DSL for headers/query/form/JSONPath
  b. Flexible: embedded startup or targeting external environments
- **Cons**:
  a. Sometimes coarse failure messages; complex assertions need extra logs
  b. Slower with full app context; limited WS/SSE support
- **References**:
  a. https://rest-assured.io/
  b. https://www.baeldung.com/rest-assured-tutorial
  c. https://reflectoring.io/rest-assured/

Method 3: Swagger UI (Manual Exploration/Contract Visualization)

- **Applicability**: Manual smoke tests during development; explore request/response shapes
- **Pros**:
  a. Zero-friction visualization of endpoints and params
  b. Self-documenting with annotations for alignment
- **Cons**:
  a. Not reusable for regression/automation
  b. Quality depends on annotations; complex auth/multi-step flows are awkward
- **References**:
  a. https://springdoc.org/
  b. https://www.baeldung.com/spring-rest-openapi-documentation
  c. https://spring.academy/courses

Method 4: Postman (Manual & Collaboration)

- **Applicability**: Smoke/debugging; shared collections/environments
- **Pros**:
  a. Pre-request scripts: auto-login to obtain JWT and set variables
  b. Friendly UI for assertions/env switching/multi-step flows
- **Cons**:
  a. Artifact drift if not versioned and CI-integrated
  b. Large-scale regression needs Newman
- **References**:
  a. https://learning.postman.com/
  b. https://blog.postman.com/tag/best-practices/

c. https://testguild.com/postman-api-testing/

Method 5: Newman (CI Runner for Postman)

- **Applicability**: Black-box regression, smoke, pre-release verification
- **Pros**:
  a. CI-friendly CLI; multiple reporters (HTML/JUnit/JSON)
  b. Reuse Postman assets
- **Cons**:
  a. Separate artifacts (code vs collections)
  b. Test data/dependency setup is less convenient than code-based tests
- **References**:
  a. https://learning.postman.com/docs/collections/using-newman-cli/
  b. https://blog.postman.com/how-to-use-newman-cli-to-run-collections-in-your-cicd-pipeline/

Method 6: Karate DSL (BDD-style Scenario Testing)

- **Applicability**: End-to-end scenarios, concurrency, data-driven tests, reporting
- **Pros**: Expressive BDD syntax; built-in concurrency and reports; mock server support
- **Cons**: DSL learning curve; less aligned with mainstream Java testing stack; weaker IDE/debug story
- **References**:
  a. https://karatelabs.github.io/karate/
  b. https://www.baeldung.com/karate-rest-api-testing
  c. https://karatelabs.io/blog

Method 7: curl / HTTPie (CLI Exploration & Scripting)

- **Applicability**: Quick checks; lightweight CI smoke; doc examples
- **Pros**: Ubiquitous; easy to script; combine with `jq`
- **Cons**: Poor readability for complex flows; weak reporting; heavy scripting for multi-step auth
- **References**:
  a. https://curl.se/docs/manual.html
  b. https://httpie.io/docs/cli
  c. https://daniel.haxx.se/blog/

d. https://www.digitalocean.com/community/cheatsheets/how-to-use-curl

3. Integration & Dependencies

Method 1: Testcontainers (Reproducible Real-Dep Tests)

- **Applicability**: Integration tests with MySQL/Redis/RocketMQ via containers
- **Pros**:
  a. High reproducibility; production-like coverage (MyBatis-Plus/transactions/cache/serialization)
  b. CI-friendly with no preinstalled services
- **Cons**:
  a. Slower due to image pulls/container startup
  b. Requires Docker and CI resource quotas
  c. RocketMQ often needs custom images or docker-compose
- **References**:
  a. https://testcontainers.com/
  b. https://www.baeldung.com/spring-boot-testcontainers
  c. https://www.testcontainers.org/blog/
  d. https://reflectoring.io/spring-boot-testcontainers/

4. External Service Stubs

Method 1: WireMock (HTTP External Service Simulation)

- **Applicability**: Simulate HTTP services like SendGrid/Microsoft Graph
- **Pros**:
  a. Deterministic and reproducible: custom error codes/latency/edge responses
  b. Offline development; contract-driven support
- **Cons**:
  a. Drift from real services over time
  b. Stub maintenance required on API changes
- **References**:
  a. https://wiremock.org/docs/
  b. https://www.baeldung.com/introduction-to-wiremock
  c. https://www.thoughtworks.com/insights/blog/service-virtualization

Method 2: MailHog / GreenMail (Email Capture/SMTP Mock)

- **Applicability**: Validate Spring Mail/SendGrid sending without real delivery
- **Pros**: Safe; visual inspection of content/headers/attachments; easy Docker setup
- **Cons**:
  a. Differences vs real ESP (scoring/rate limit/delivery policies)
  b. Webhooks/events need a separate simulation
- **References**:
  a. https://github.com/mailhog/MailHog
  b. https://www.icegreen.com/greenmail/
  c. https://www.baeldung.com/spring-email

5. Performance & Reliability

Method 1: k6 (Developer-friendly Performance Testing)

- **Applicability**: Smoke/stress/soak/stability tests
- **Pros**: JS scripting with programmable assertions; efficient resource usage; CI + Grafana integration
- **Cons**: Weak GUI; complex protocol/transaction modeling requires custom code
- **References**:
  a. https://k6.io/docs/
  b. https://grafana.com/blog/tags/k6/
  c. https://testguild.com/k6/

Method 2: JMeter (Mature GUI Load Testing)

- **Applicability**: Concurrency/parameterization/record-replay/plugin ecosystem
- **Pros**: Feature-rich; friendly for non-programmers; many plugins
- **Cons**: Maintainability issues for large XML plans; higher resource usage
- **References**:
  a. https://jmeter.apache.org/
  b. https://www.blazemeter.com/blog
  c. https://dzone.com/jmeter-tutorials-tools-news

Method 3: Gatling (Scala DSL, High Performance)

- **Applicability**: High-concurrency, programmable load tests

- **Pros**: High throughput; code-as-tests; clear HTML reports
- **Cons**: Scala learning curve; auth preparation requires wrappers
- **References**:
    a. https://gatling.io/docs/
    b. https://gatling.io/blog/
    c. https://www.baeldung.com/gatling

6. Security

Method 1: OWASP ZAP (Dynamic App Security Testing)

- **Applicability**: Baseline/active scans for XSS/SQLi/CORS, etc.
- **Pros**: Free/open-source; CI-ready; scriptable auth and contexts
- **Cons**: False positives/negatives require human triage; complex auth/user flows are costly to model
- **References**:
    a. https://www.zaproxy.org/docs/
    b. https://www.zaproxy.org/blog/
    c. https://github.com/zaproxy/action-baseline
    d. https://portswigger.net/daily-swig